School of Future Tech

Case Study Report

on

Case Study 35. Expense Tracking & Budget Planner

by

Aman Ashok Koli

150096725119

1. Introduction to the Case Study

Effective financial management is a crucial skill for personal stability. However, many individuals struggle to keep a clear record of where their money goes. This case study focuses on designing and implementing a C++ console application to serve as an Expense Tracking & Budget Planner. The system utilizes Object-Oriented Programming (OOP) concepts to help users automatically track their daily expenses and plan their budgets effectively. It highlights the importance of recording data, calculating totals, and providing real-time financial summaries to the user.

2. Problem Statement / Case Background (Abstract)

Background

Poor expense tracking often leads to financial mismanagement. Without a structured way to record expenditures, individuals can easily overspend and exceed their available funds.

Objectives

The primary objective of this system is to manage personal finances by tracking expenses and planning budgets.

Abstract

This case study presents the design and development of an Expense Tracking system using C++. The application satisfies the core functional requirements of budget setting, expense recording, and generating summary reports. By utilizing classes, arrays, and standard control structures, the software records individual expense entries (including name, category, and amount) and compares the total expenditure against a predefined monthly budget. The final deliverable is a functional budget planning software that warns users if they exceed their budget.

3. Case Study Design

The Expense Tracker system is designed as an interactive, menu-driven application with the following core modules:

- **Budget Setting:** The system allows the user to input a total monthly budget.

- **Expense Recording:** The user can add individual expenses by providing the expense name, category (e.g., Food, Travel), and the monetary amount. These are stored in an array of objects.

- **Data Display:** The system formats and prints a list of all recorded expenses.

- **Summary Reports:** The software calculates the total expenses, calculates the remaining budget, and triggers a warning message ("⚠ Warning: Budget Exceeded!") if the expenses surpass the set budget limit.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Key Methods and Algorithms:

- **Object-Oriented Design:** The system uses a data class (Expense) to structure the data and a manager class (ExpenseTracker) to handle the logic and state (budget, expense count).
- **Array Management:** An array of objects (Expense expenses[100]) is used to store up to 100 expense entries in memory.
- **Iterative Accumulation:** A for loop algorithm is used within the showSummary() method to continuously add the amounts of all recorded expenses to calculate the total spent.
- **Control Flow:** A do-while loop combined with a switch-case statement ensures the program menu runs continuously until the user explicitly chooses to exit.

Technology Stack Used for the Case

- Programming language: C++
- Libraries: <iostream> for input/output streams, <string> for text handling.
- Environment: C++ Compiler (e.g., GCC) / IDE (e.g., VS Code, Dev C++, Code::Blocks).

5. Problem Statement / Case Study Implementation Details and Snapshots.

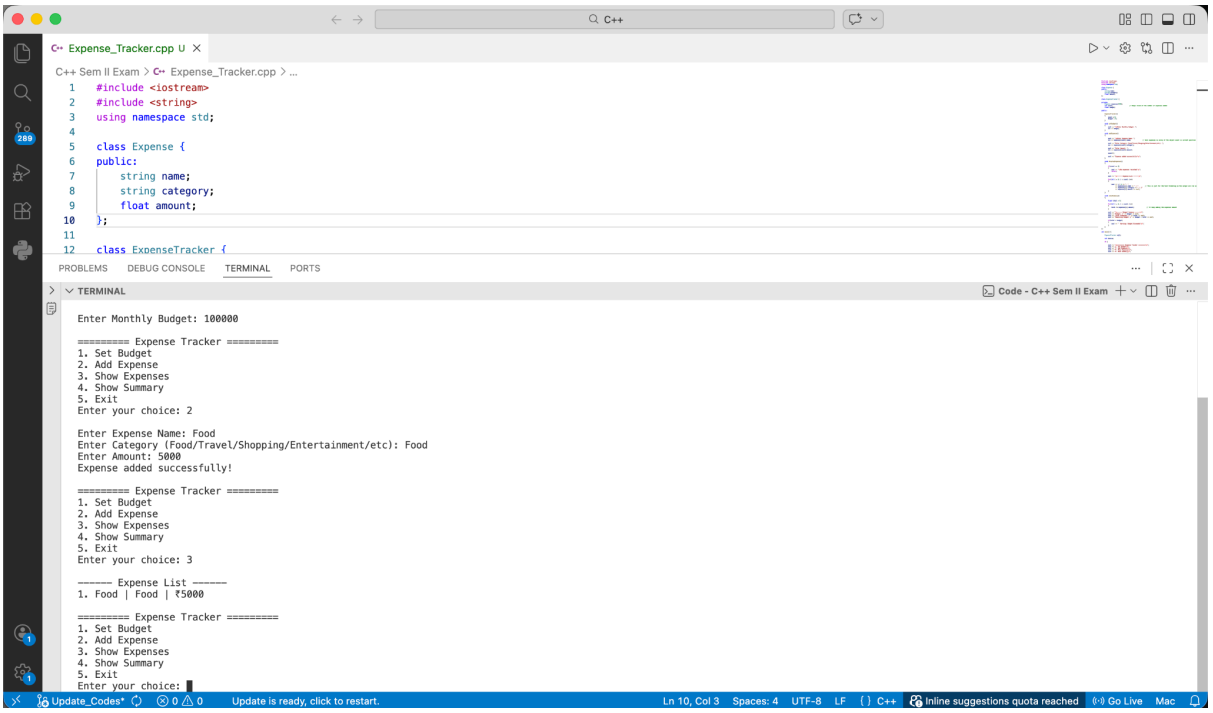
The implementation revolves around two primary C++ classes:

- class Expense: Acts as a data container holding name, category, and amount.
- class ExpenseTracker: Contains the operational logic. It manages the expenses array, an integer count to track the current array index, and a float budget.

The main() function instantiates ExpenseTracker obj1; and uses a switch statement to call functions like setBudget(), addExpense(), displayExpenses(), and showSummary() based on user input.

Snapshots:

1. Setting the budget and adding a few expenses.



2.Displaying the expense list.

```
========= Expense Tracker =========
1. Set Budget
2. Add Expense
3. Show Expenses
4. Show Summary
5. Exit
Enter your choice: 3

------ Expense List ------
1. Food | Food | ₹5000
```

3. Showing the summary report

```
========= Expense Tracker =========
1. Set Budget
2. Add Expense
3. Show Expenses
4. Show Summary
5. Exit
Enter your choice: 4

------ Budget Summary ------
Budget: ₹100000
Total Expenses: ₹5000
Remaining Budget: ₹95000

--------- Expense Tracker ---------
```

6. Problem Statement / Case Study Results and Conclusion.

Findings

The application successfully operates as a digital ledger. It was observed that utilizing an array of objects is an efficient way to temporarily store relational data (name, category, amount) in a basic console application without requiring a database.

Conclusion

The development of the Expense Tracking & Budget Planner fulfills all required deliverables. The system effectively handles expense recording, budget setting, and summary report generation. By providing an immediate calculation of total expenses and remaining funds, alongside a budget warning, this software directly addresses the problem of financial mismanagement caused by poor expense tracking.

7. References

- ITM Skills University Case Study Requirements: Case Study 35. Expense Tracking & Budget Planner.
- C++ Standard Library Documentation for <iostream> and <string>.